



OMNIX

Decision Governance Infrastructure

Velos Integration Document | Schema v6.5.4e | T=0 Gateway Edition

For	Naimat Khan — Velos
Prepared by	Harold Nunes — OMNIX QUANTUM LTD
Date	April 09, 2026
Classification	Confidential — Partner Integration Material
Schema Version	v6.5.4e
Signing Standard	NIST FIPS 204 — ML-DSA (Dilithium-3 / Dilithium-5)
KEM Standard	NIST FIPS 203 — ML-KEM (Kyber-768)
Endpoint	POST https://velos-gateway.onrender.com/api/v1/intercept

Every OMNIX decision produces a cryptographically signed JSON receipt encoding the complete governance trail. This document defines the exact schema Velos must implement to verify receipts at the T=0 interceptor gateway — including TTL enforcement, hash integrity, and PQC signature dispatch.

1. Complete JSON Schema

```
{
  "receipt_id": "OMNIX-A3F7C1D92B0E",
  "timestamp": "2026-04-09T14:23:01.847291+00:00",
  "issued_at_ms": 1744208581847, // ← T=0 reference (epoch ms)
  "ttl_epoch_ms": 1744208611847, // ← expiry (epoch ms). REJECT if now_ms > this
  "ttl_ms": 30000, // ← default 30 s, configurable
  "asset": "BTC/USDT",
  "decision": "BLOCK", // BLOCK | ALLOW | PASS | UNKNOWN
  "veto_chain": ["AML: PASS ...", "SHARIA: BLOCK -- gharar=0.81 ..."],
  "policy_version": "6.5.4e",
  "engine_version": "6.5.4e",
  "prev_hash": "a9f3c1d4...", // SHA-256 of previous receipt (chain)
  "signing_provider": "dilithium3", // dilithium3 | dilithium5 | sha256
  "content_hash": "3f8e7a2c...", // SHA-256 of canonical payload (signed)
  "signature": "BASE64==...", // encoding → see signature_format
  "signature_algorithm": "Dilithium-3 (ML-DSA-65)",
  "signature_format": "base64_pqc", // base64_pqc | hex_sha256_fallback | NONE
  "public_key": "BASE64==...", // null in fallback mode
  // Conditional – present only when gate was evaluated:
  "sharia_compliance": {...}, "aml_compliance": {...}, "fraud_compliance": {...},
  "jurisdiction_compliance": {...}, "context_admission": {...}, "avm_result": {...}
}
```

2. Field Reference

Field	Type	Description
receipt_id	string	Globally unique ID. Format: OMNIX-{12 hex uppercase}
timestamp	string	ISO-8601 UTC generation time
issued_at_ms	integer	Unix epoch ms — canonical T=0 reference point
ttl_epoch_ms	integer	Expiry in epoch ms. Velos MUST reject if now_ms > ttl_epoch_ms
ttl_ms	integer	TTL in ms. Default 30000 (30 s). Set via OMNIX_RECEIPT_TTL_MS
asset	string	Asset symbol. e.g. BTC/USDT, ETH/USD
decision	string	BLOCK ALLOW PASS UNKNOWN (uppercased)
veto_chain	array	Ordered gate verdicts. Format: 'GATE: RESULT — detail'
policy_version	string	Policy version used for this evaluation
engine_version	string	OMNIX engine version (current: 6.5.4e)
prev_hash	string	SHA-256 of previous receipt — tamper-evident chain. " if first
signing_provider	string	Active algorithm ID bound into signed payload (anti-downgrade)
content_hash	string	SHA-256 hex of canonical JSON of all signed fields
signature	str null	Detached signature over content_hash. Encoding = signature_format
signature_algorithm	string	Human-readable. e.g. 'Dilithium-3 (ML-DSA-65)', 'SHA-256'
signature_format	string	CRITICAL. base64_pqc → PQC bytes (base64). hex_sha256_fallback → symmetric hex. NONE → reject.
public_key	str null	Base64 PK for asymmetric verification. null in fallback mode

3. signature_format Dispatch

signature_format	signature encoding	How Velos verifies
base64_pqc	Base64-encoded raw PQC bytes	base64.decode → Dilithium.verify(sig, content_hash, public_key)
hex_sha256_fallback	Hex of SHA-256(content_hash). Symmetric. public_key = null	sha256(content_hash.encode()).hexdigest() == signature
NONE	Signing failed — no integrity guarantee	REJECT unconditionally

4. Verification Algorithm (drop-in Python)

```
import json, hashlib, base64, time

def verify_omnix_receipt(receipt):
    # 1 – TTL
    ttl = receipt.get('ttl_epoch_ms')
    if ttl and int(time.time()*1000) > ttl:
        return {'valid': False, 'reason': 'RECEIPT_EXPIRED'}

    # 2 – Rebuild payload for hash
    MANDATORY = ['receipt_id', 'timestamp', 'issued_at_ms', 'ttl_epoch_ms', 'ttl_ms',
        'asset', 'decision', 'veto_chain', 'policy_version', 'engine_version',
        'prev_hash', 'signing_provider']
    CONDITIONAL = ['sharia_compliance', 'aml_compliance', 'fraud_compliance',
        'jurisdiction_compliance', 'context_admission', 'veto_type', 'avm_result']
    p = {k: receipt[k] for k in MANDATORY if k in receipt}
    for k in CONDITIONAL:
        if k in receipt: p[k] = receipt[k]

    # 3 – Verify content_hash
    computed = hashlib.sha256(json.dumps(p, sort_keys=True,
        ensure_ascii=True).encode()).hexdigest()
    if computed != receipt.get('content_hash'):
        return {'valid': False, 'reason': 'HASH_MISMATCH'}

    # 4 – Verify signature
    fmt = receipt.get('signature_format', 'UNKNOWN')
    sig = receipt.get('signature')
    msg = receipt['content_hash'].encode()
    if fmt == 'base64_pqc':
        prov = receipt.get('signing_provider', 'dilithium3')
        if prov == 'dilithium3':
            from pqc.sign import dilithium3
            try: dilithium3.verify(base64.b64decode(sig), msg,
                base64.b64decode(receipt['public_key']))
            except: return {'valid': False, 'reason': 'SIGNATURE_INVALID'}
        elif prov == 'dilithium5':
            from pqc.sign import dilithium5
            try: dilithium5.verify(base64.b64decode(sig), msg,
                base64.b64decode(receipt['public_key']))
            except: return {'valid': False, 'reason': 'SIGNATURE_INVALID'}
        elif fmt == 'hex_sha256_fallback':
            if hashlib.sha256(receipt['content_hash'].encode()).hexdigest() != sig:
                return {'valid': False, 'reason': 'FALLBACK_HASH_MISMATCH'}
            else:
                return {'valid': False, 'reason': f'UNKNOWN_FORMAT:{fmt}'}
    return {'valid': True}
```

5. Cryptographic Assurance Tiers

Tier	signing_provider	Algorithm	Standard	Security	PK	Sig
Enterprise (default)	dilithium3	Dilithium-3 (ML-DSA-65)	FIPS 204	~192-bit	1952 B	~3309 B

High-Assurance	dilithium5	Dilithium-5 (ML-DSA-87)	FIPS 204	~256-bit	2592 B	~4627 B
No PQC Available	sha256	SHA-256 symmetric fallback	—	N/A	N/A	64 hex

6. content_hash Contract — What Gets Hashed

Mandatory (always in hash):

asset, decision, engine_version,
issued_at_ms, policy_version, prev_hash,
receipt_id, signing_provider, timestamp,
ttl_epoch_ms, ttl_ms, veto_chain

Conditional (if present):

sharia_compliance, aml_compliance,
fraud_compliance, jurisdiction_compliance,
context_admission, veto_type, avm_result

NOT in hash (detached pattern):

content_hash, signature,
signature_algorithm,
signature_format, public_key

7. Integration Checklist

#	Check	Rule
1	now_ms > ttl_epoch_ms?	Reject immediately with RECEIPT_EXPIRED
2	Read signature_format	Never hardcode — dispatch on this field
3	Read signing_provider	Determines PQC algorithm for verification
4	pip install pypqc	Required on Velos side for PQC verification
5	Recompute content_hash	Exclude: signature, public_key, sig_format, sig_algorithm, content_hash
6	decision == BLOCK?	Do not proceed — OMNIX has vetoed the action
7	signature_format == NONE?	Reject unconditionally — no integrity guarantee

Harold Nunes — OMNIX QUANTUM LTD | Eureka GCC Dubai 2026 Semifinalist | Raising \$500K pre-seed @ \$3M valuation